

SupplyGuard AI

Autonomous Supply Chain Disruption Copilot & Fleet Utilisation Optimizer

Track L2 — Supply Chain Disruption Assistant & Fleet Utilisation Optimizer

■ Team LEO

Logistics · Engineering · Optimization

Meet Nakrani · 23DCS064

Akshit Vaghasiya · 23DIT075

Contents

1. Team & Identity	3
2. Problem Statement	3
3. Proposed Solution	4
4. System Architecture	5
5. Technology Stack	7
6. Data Model	8
7. MCP Server & Tools	9
8. Application Modules (Operational Workspaces)	10
9. AI Copilot — End-to-End Workflow	11
10. Security, RBAC & Governance	12
11. Setup & Installation Guide	12
12. Testing Strategy	13
13. Business Value & Measurable Impact	13
14. IBM Bob Integration	14
15. Known Limitations & Future Scope	14
16. Conclusion	15
17. Team Contribution	15

1. Team & Identity

Our team name, LEO, is a deliberate acronym that maps directly onto what this project actually does:

Letter	Stands for	Maps to
L	Logistics	Supply chains, shipments, ports, fleet
E	Engineering	MERN stack, Python, MCP, AI, system architecture
O	Optimization	Rerouting, fleet utilization, risk reduction

Name	Enrollment No.	Primary Focus
Meet Nakrani	23DCS064	Engineering — MERN stack (React, Node.js, Express, MongoDB), MCP server & tool implementation, system architecture
Akshit Vaghasiya	23DIT075	Engineering — Python analytics service (FastAPI, NumPy/Pandas), documentation & presentation

2. Problem Statement

Global maritime and intermodal commerce is the backbone of the global economy, yet it remains critically fragile. When an acute disruption strikes a major node — an unannounced dockworker strike, a geopolitical chokepoint blockade, or a cold-chain equipment failure — supply chain operators face five systemic breakdowns:

- **Information overload & conflicting intelligence** — operators receive disjointed notices from port authorities, news wires, and maritime bulletins; separating authentic crises from rumours takes hours of manual cross-checking.
- **Cold-chain spoilage risk** — temperature-sensitive biopharmaceuticals and vaccines can suffer irreversible excursions during port congestion; once disconnected from shore-power reefer plugs, containers have only a few hours of battery buffer before high-value cargo is destroyed.
- **Demurrage cascades** — a halted berth backs up queues across feeder routes, generating per-day vessel demurrage fees, missed SLA penalties, and downstream delays.
- **Fleet asset misallocation** — vessels and trucks idle outside congested ports while nearby secondary hubs and dedicated fleet assets sit unused due to lack of synchronized visibility.
- **Decision paralysis & governance void** — logistics managers must weigh transit time vs. cost vs. spoilage risk under pressure, with decisions made over chat/email leaving no auditable trail.

Who this affects: logistics managers, operations analysts, and fleet dispatchers at freight and shipping companies managing large numbers of active shipments across ports and carriers.

3. Proposed Solution

SupplyGuard AI replaces manual, reactive firefighting with an explainable, human-approved decision loop:

1. **Verify** — cross-check disruption reports against multiple sources and assign a confidence score instead of acting on rumour.

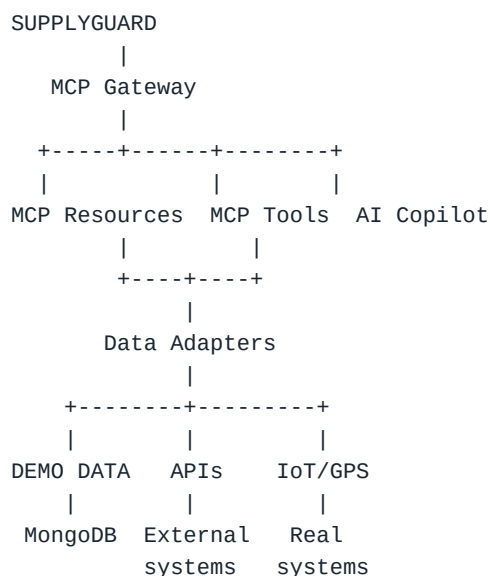
2. **Identify impact** — automatically find every shipment, port, and fleet asset affected by the disruption.
3. **Protect cold-chain cargo** — monitor temperature telemetry in real time and flag excursions before spoilage occurs.
4. **Compare options** — generate ranked alternative plans (reroute, redeploy fleet, hold) with cost/time/risk trade-offs.
5. **Human approval** — no action executes automatically; a Logistics Manager or Admin must approve it.
6. **Audit** — every approved action is permanently logged for compliance and accountability.

Core design principle: the AI recommends, a human decides, and every decision is recorded. This is what separates SupplyGuard AI from a black-box automation tool.

4. System Architecture

SupplyGuard AI is built around the Model Context Protocol (MCP) as a data-integration layer between the AI copilot and every underlying capability. The same MCP interface works whether the data behind it is synthetic (hackathon) or a real enterprise system (production) — only the adapter underneath changes.

High-Level MCP Structure



During the Hackathon (Synthetic + Public Data In)

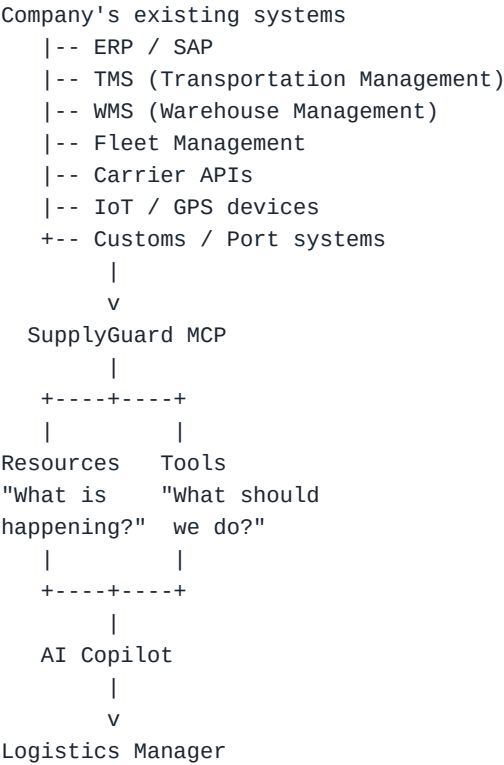
```

Synthetic shipments -----+
Synthetic fleet -----|
Synthetic GPS -----|--> SupplyGuard
Synthetic IoT -----|
Real/public news -----|
Real port information --+
  
```

In the Real World (Adapters Swapped for Production Systems)

```
Company TMS -----+
Company ERP -----|
Fleet GPS -----|--> MCP Gateway --> SupplyGuard
IoT sensors -----|
Port systems -----|
Carrier APIs -----|
Weather/events -----+
```

End-to-End Real-World Data Flow



Think of MCP as a data integration layer: Resources answer “what is happening?”, Tools answer “what should we do?” — the AI Copilot combines both to produce a recommendation for the human decision-maker.

Component Responsibilities

Layer	Technology	Responsibility
Frontend	React 18 + TypeScript + Vite + TailwindCSS	Operator dashboard, maps, risk studio, copilot chat UI
Backend	Node.js + Express + TypeScript	REST API, JWT auth, RBAC, MCP server host
Analytics	Python 3 + FastAPI + NumPy/Pandas	Risk scoring, routing, cold-chain thermal modelling, simulation
Persistence	MongoDB (Mongoose) + Redis	Operational data storage + caching
AI Layer	Model Context Protocol + Claude API	Tool-calling orchestration for the AI Copilot

5. Technology Stack

React 18 · TypeScript · Vite · TailwindCSS · Leaflet (GIS Map) · Node.js · Express · MongoDB + Mongoose · Python 3 + FastAPI · NumPy / Pandas · Redis · Model Context Protocol SDK · Anthropic Claude API · JWT + RBAC · Docker Compose · GitHub Actions · IBM BoB

The stack was chosen so that each layer has one clear job: React for the operator experience, Express for orchestration and access control, FastAPI/Python for numeric analytics, and MCP as the strict contract layer between the AI and everything else — making the system easy to reason about and to extend with real enterprise connectors later.

6. Data Model

Collection	Key fields
users	user_id, organization_id, role, permissions
organizations	organization_id, name, tenant_settings
ports	port_id, name, country, location, capacity, congestion, status
shipments	shipment_id, origin, destination, route, cargo_type, value, weight, carrier, deadline, priority, status
fleet	vehicle_id, asset_type, location, capacity, status, availability, refrigeration_capable
routes	route_id, origin, destination, distance, estimated_time, cost, risk_score, capacity
carriers	carrier_id, name, capacity, routes, reliability_score, availability, cost
disruptions	disruption_id, type, location, severity, start_time, expected_end, affected_routes, evidence
news_events	event_id, source, published_at, normalized_event, credibility, verification_status
temperature_logs	container_id, timestamp, temperature, min_threshold, max_threshold
port_events	port_id, timestamp, event_type, severity, details
audit_logs	actor, organization_id, action, target, reason, timestamp, approval_status

7. MCP Server & Tools

The MCP server is the core of the AI layer. It exposes 10 tools and 9 resources that the AI Copilot calls to answer operator questions with real, verifiable data rather than guesses.

Tool	Purpose
verify_disruption	Validates incident authenticity via multi-source corroboration and assigns a confidence score
find_affected_shipments	Identifies in-transit shipments entering the disruption impact zone
calculate_shipment_risk	Evaluates a multi-factor risk score combining perishability, delay, and congestion
find_alternative_ports	Discovers diversion ports within reach, ranked by reefer plug availability and capacity
evaluate_route	Calculates distance, cost delta, and transit time for a candidate reroute
find_available_fleet	Scans idle trucks, rail, and vessels available for mobilization
analyze_cold_chain	Simulates thermal degradation and predicts remaining safe buffer hours
simulate_network_impact	Projects the 24h/72h financial cascade of the disruption
compare_scenarios	Generates a side-by-side trade-off matrix across candidate plans
generate_action_plan	Synthesizes the final operational plan with approval sign-off

Resources (read-only context): port://{id}/status, port://{id}/congestion, port://{id}/disruptions, shipments://active, shipments://at-risk, fleet://available, routes://risk, coldchain://alerts, news://logistics/disruptions.

8. Application Modules (Operational Workspaces)

Workspace	What it does
Mission Control	Executive dashboard — active disruptions, cargo exposure, shipments monitored, cold-chain status
Threat Intel & Ports	Verifies whether a reported disruption is real, with a confidence score and evidence sources
Cold-Chain Guard	Live temperature telemetry for refrigerated cargo; triggers emergency truck intercept before spoilage
Scenario Matrix	Side-by-side comparison of alternative response plans (cost, transit time, risk)
Tactical Maritime Map	Visual map of ports, vessels, and routes (Leaflet GIS)
Fleet & Cargo Manifest	Full shipment/fleet list with per-item risk scores and one-click mitigation
72h Cascade Simulator	Projects the financial cost of inaction over the next 72 hours

Workspace	What it does
Governance & Audit Trail	Immutable log of every approved action, actor, and justification
AI Operations Copilot	Natural-language chat interface that calls the MCP tools automatically
Operations & RBAC Guide	In-app reference for role permissions
Admin Personnel Console	User/role management

9. AI Copilot — End-to-End Workflow

Example query: *"Mumbai Port has a strike. What should I do?"*

1. Read disruption/port MCP resources
2. Verify evidence (verify_disruption)
3. Find affected shipments
4. Calculate shipment risk
5. Check cold-chain exposure
6. Find alternative ports/routes
7. Check available fleet
8. Simulate 24h/72h impact
9. Compare scenarios (Plan A/B/C)
10. Generate explainable action plan
11. Request human approval for major action
12. Record audit event
13. Show before/after metrics

Every step's tool call and result is logged with a correlation ID, so a manager can trace exactly how a recommendation was produced — this is what makes the copilot explainable rather than a black box.

10. Security, RBAC & Governance

- JWT access + refresh tokens, bcrypt-hashed passwords
- Role-based access control: Logistics Manager (full operational execution), Admin (global policy & user configuration), Supply Chain Analyst (analytical/read-only), Auditor (compliance review)
- Organization-scoped data isolation on every query
- No major action (reroute, fleet redeployment) executes without human approval
- Every approved action is written to an append-only audit log with actor, timestamp, and justification

11. Setup & Installation Guide

```
git clone https://github.com/[team]/bob-ai-hackathon-team-leo.git
cd bob-ai-hackathon-team-leo
cp src/.env.example src/.env    # fill in local values, never commit this file
docker-compose up --build

# in a second terminal, seed the database
docker-compose exec backend npm run seed

# verify each service
curl http://localhost:5000/api/health    # backend
curl http://localhost:8000/health        # python analytics service
curl http://localhost:7331/health        # MCP server (HTTP mode)

# open the app
open http://localhost:5173                # frontend
```

12. Testing Strategy

Layer	Tooling	Coverage target
Backend unit tests	Jest	Risk/routing/fleet logic
Backend API tests	Supertest	Every endpoint: success + auth + validation failure
Python analytics tests	Pytest	Scoring/optimization modules with deterministic fixtures
MCP contract tests	MCP Inspector	Every tool's input/output validated against schema
CI gate	GitHub Actions	All suites must pass before merge to main

13. Business Value & Measurable Impact

Metric	Traditional manual process	SupplyGuard AI
Disruption verification time	4–8 hours of phone calls and cross-checking	Seconds, via multi-source corroboration
Cold-chain spoilage risk	Discovered only at delivery, too late to act	Detected in real time, before delivery
Alternative route evaluation	Manual spreadsheets, siloed teams	Instant multi-factor trade-off comparison
Regulatory compliance & audit	Fragmented email threads	Structured, timestamped audit ledger

Figures shown in the project pitch (e.g. specific dollar amounts) are illustrative demo-scenario values built on synthetic data, not measured production results.

14. IBM Bob Integration

IBM Bob was used as a load-bearing part of the build process:

- **Plan mode** — turned this specification into the phased task breakdown used to build the app
- **Agent mode** — implemented core parts of the backend and the AI Copilot orchestration logic
- **Review** — code review pass on the MCP server and backend before merging
- **Diagram generation** — architecture diagrams drawn from the real codebase

15. Known Limitations & Future Scope

Known limitations

- Uses synthetic/demo data for shipments, fleet, ports, and disruptions — clearly labeled as simulated, not live GPS/AIS or production feeds
- Cold-chain and disruption detection use rule-based scoring rather than trained ML models, to keep results explainable within the project timeframe
- No production-grade secrets management (local .env only)

Future scope

- Replace synthetic adapters with real TMS/ERP, GPS/AIS, IoT, and carrier/port system connectors — the MCP boundary is designed so this swap doesn't change the decision layer
- Add ML-based risk scoring once sufficient historical data is available, while keeping the rule-based engine as an explainable fallback
- Production security hardening: managed secrets, enterprise identity/SSO, observability, disaster recovery

16. Conclusion

SupplyGuard AI demonstrates how the Model Context Protocol can ground an AI copilot in verifiable operational tools and real-time telemetry, while keeping a human logistics manager firmly in control of every high-impact decision. By combining evidence-based disruption verification, cold-chain protection, fleet optimization, and an immutable audit trail into one connected workflow, the project shows a practical, explainable, and safe approach to autonomous decision support in supply chain operations.

17. Team Contribution

Name	Enrollment No.	Contribution
Meet Nakrani	23DCS064	MERN stack implementation (frontend + backend), MCP server & tool implementation, system architecture, IBM Bob-driven development
Akshit Vaghasiya	23DIT075	Python analytics service (risk scoring, routing, cold-chain thermal modelling), UI/UX design input, documentation, and presentation